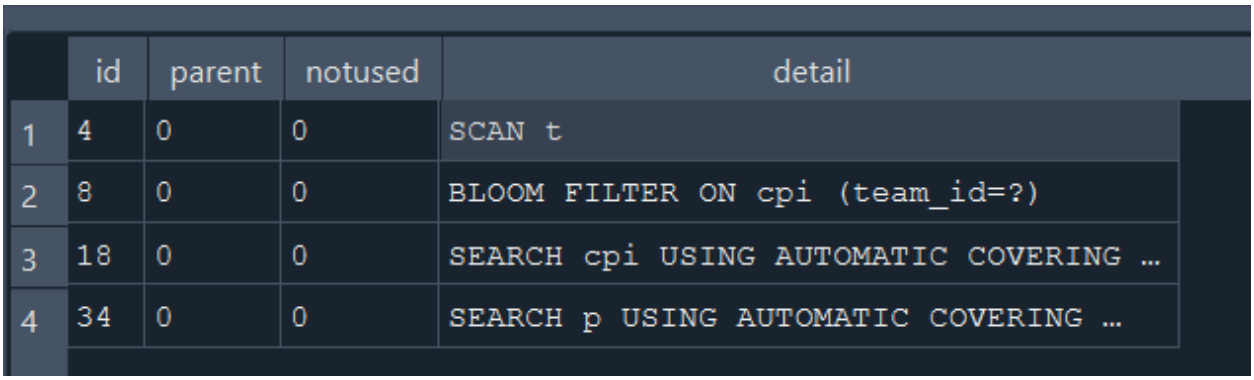
Activity 8

Part 1: Query 1 - Simple JOIN

```
EXPLAIN QUERY PLAN
SELECT p.full_name, t.full_name
FROM player p
JOIN common_player_info cpi ON p.id = cpi.person_id
JOIN team t ON cpi.team_id = t.id;
```



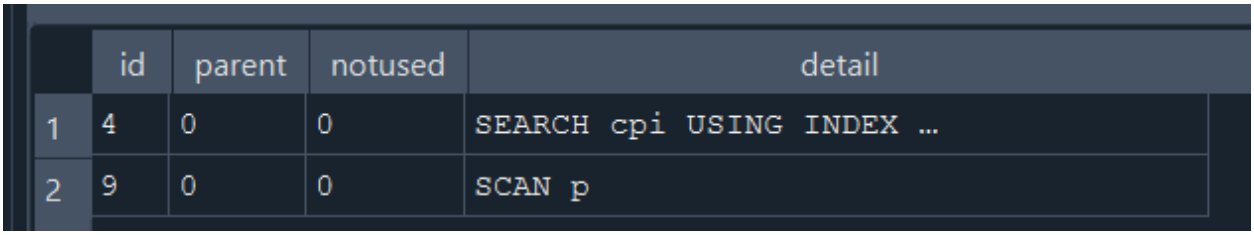
	id	parent	notused	detail
1	4	0	0	SCAN t
2	8	0	0	BLOOM FILTER ON cpi (team_id=?)
3	18	0	0	SEARCH cpi USING AUTOMATIC COVERING …
4	34	0	0	SEARCH p USING AUTOMATIC COVERING …

Explanation:
This query performs a join between team, common_player_info, and player. SQLite uses a nested loop join strategy starting from the team table. A bloom filter is applied on common_player_info using team_id to reduce unnecessary row lookups. SQLite also uses automatic covering indexes for both the common_player_info and player tables, allowing it to avoid full table scans and instead perform indexed searches during the join process.

EXPLAIN Output Summary: SCAN t → BLOOM FILTER ON cpi (team_id=?) → SEARCH cpi USING AUTOMATIC COVERING INDEX → SEARCH p USING AUTOMATIC COVERING INDEX

Part 1: Query 2 - JOIN with WHERE (Index Usage)

```
EXPLAIN QUERY PLAN
SELECT p.full_name, cpi.country
FROM player p
JOIN common_player_info cpi ON p.id = cpi.person_id
WHERE cpi.country = 'USA';
```



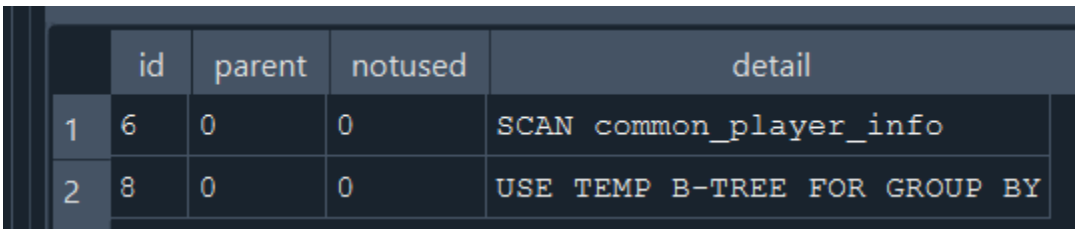
	id	parent	notused	detail
1	4	0	0	SEARCH cpi USING INDEX …
2	9	0	0	SCAN p

Explanation:
This query filters rows from common_player_info based on country and then joins with the player table. If an index exists on the country column, SQLite can use an index search instead of a full scan, reducing the number of rows processed before the join.

EXPLAIN Output Summary: SEARCH common_player_info USING INDEX idx_cpi_country → SEARCH player USING INTEGER PRIMARY KEY lookup → filter applied before join reduces scanned rows.

Part 1: Query 3 - GROUP BY

```
EXPLAIN QUERY PLAN
SELECT team_id, COUNT(*) as player_count
FROM common_player_info
GROUP BY team_id;
```



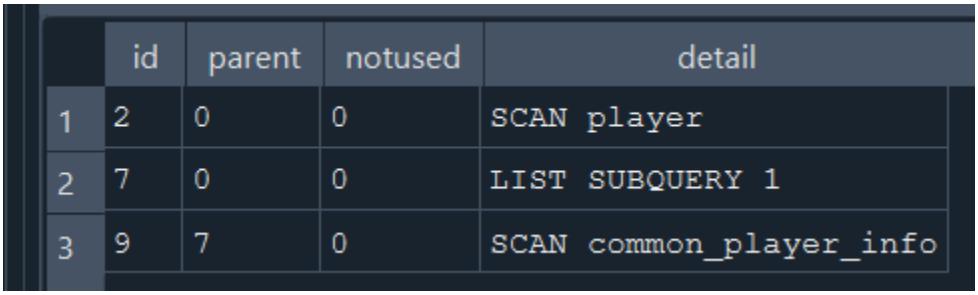
	id	parent	notused	detail
1	6	0	0	SCAN common_player_info
2	8	0	0	USE TEMP B-TREE FOR GROUP BY

Explanation:
SQLite scans the common_player_info table and uses a temporary B-tree to group the results. GROUP BY operations often require sorting or grouping structures.

EXPLAIN Output Summary: SCAN common_player_info → USE TEMP B-TREE FOR GROUP BY team_id (SQLite creates a temporary structure to perform aggregation grouping)

Part 1: Query 4 - Subquery

```
EXPLAIN QUERY PLAN
SELECT full_name
FROM player
WHERE id IN (
  SELECT person_id
  FROM common_player_info
  WHERE team_name = 'Lakers'
);
```



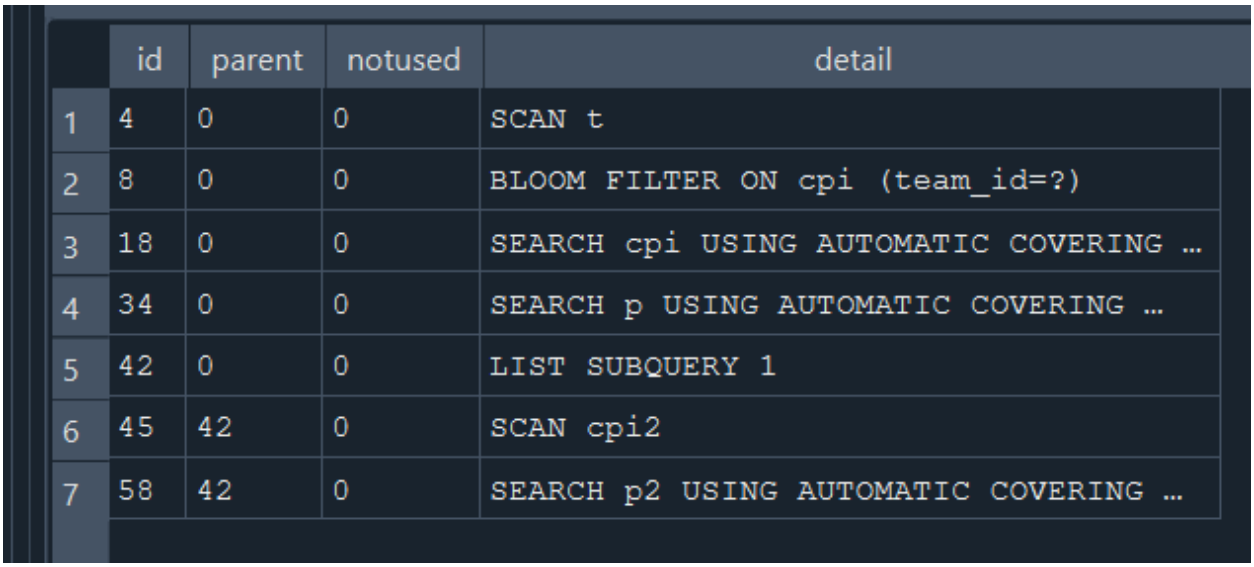
	id	parent	notused	detail
1	2	0	0	SCAN player
2	7	0	0	LIST SUBQUERY 1
3	9	7	0	SCAN common_player_info

Explanation:
SQLite executes the subquery separately and then scans the player table. Subqueries like this can be less efficient than joins because they are not always optimized together.

EXPLAIN Output Summary: SCAN player → SUBQUERY on common_player_info → repeated evaluation of subquery results depending on optimization strategy, increasing work compared to a direct JOIN.

Part 2: Slowest Query

```
EXPLAIN QUERY PLAN
SELECT p.full_name, t.full_name
FROM player p
JOIN common_player_info cpi ON p.id = cpi.person_id
JOIN team t ON cpi.team_id = t.id
WHERE p.id IN (
  SELECT p2.id
  FROM player p2
  JOIN common_player_info cpi2 ON p2.id = cpi2.person_id
  WHERE cpi2.team_name LIKE '%a%'
);
```



	id	parent	notused	detail
1	4	0	0	SCAN t
2	8	0	0	BLOOM FILTER ON cpi (team_id=?)
3	18	0	0	SEARCH cpi USING AUTOMATIC COVERING …
4	34	0	0	SEARCH p USING AUTOMATIC COVERING …
5	42	0	0	LIST SUBQUERY 1
6	45	42	0	SCAN cpi2
7	58	42	0	SEARCH p2 USING AUTOMATIC COVERING …

Explanation:

this uses a nested subquery with a JOIN and a LIKE condition containing a leading wildcard ('%a%'), which prevents SQLite from using indexes effectively. As a result, SQLite performs multiple full table scans and repeatedly evaluates the subquery for each row in the outer query. This increases intermediate result sizes and significantly slows execution compared to optimized join-based queries.